DSCI-560: Data Science Practicum Laboratory Assignment 10

AI Enhanced Group Chat Web and Android App

Instructor: Young H. Cho, Ph.D.

Prerequisites

1. OS: Linux, macOS, or Windows
2. MySQL 8+
3. Python 3.10+ and pip
4. Node.js + npm if you plan to use the bubblewrap CLI
5. Android Studio Flamingo+ (or newer), JDK 17, Android SDK

Part 1: Web Application

I. Templates

1. Download the ZIP file for this Assignment
2. Review the README file and follow the instructions

II. MySQL: Install & Database Setup

1. Install MySQL Server (consult your OS package manager or installer).
2. Start MySQL and secure it (set root password).
3. Create database and user (or run sql/schema.sql):

```
CREATE DATABASE groupchat CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'chatuser'@'localhost' IDENTIFIED BY 'chatpass';
GRANT ALL PRIVILEGES ON groupchat.* TO 'chatuser'@'localhost';
FLUSH PRIVILEGES;
```

III. Backend (FastAPI) Setup

```
cd backend
python -m venv .venv
source .venv/bin/activate   (Windows: .venv\Scripts\activate)
pip install -r requirements.txt
```

1. Copy .env.example to `../.env` and edit values (DB DSN, JWT secret, LLM endpoint)
2. Run the server: `uvicorn app:app --host 0.0.0.0 --port 8000`
3. Open `http://localhost:8000` in a browser.
4. Sign up, then log in. Messages with a "?" trigger the LLM bot.

IV. Configuration (`.env`)

1. Edit .env using an editor like gedit/nano/vim.
2. Then copy paste the bellow lines and save the file.

```
DATABASE_URL=mysql+asyncmy://chatuser:chatpass@localhost:3306/groupchat
JWT_SECRET=<replace with a long random string>
JWT_EXPIRE_MINUTES=43200
LLM_API_BASE=http://localhost:8001/v1
```

```
LLM_MODEL=llama-3-8b-instruct
LLM_API_KEY=
APP_HOST=0.0.0.0
APP_PORT=8000
```

V. Self-Hosted Llama 3 Endpoint (Option A)

1. Build llama.cpp:

```
git clone https://github.com/ggerganov/llama.cpp
cd llama.cpp
cmake -B build
cmake --build build --config Release
```

2. Obtain Llama 3 weights from Meta/Hugging Face. Accept the license. Download the model.
3. One option: https://huggingface.co/bartowski/Meta-Llama-3.1-70B-Instruct-GGUF
4. (If needed) Convert to GGUF using llama.cpp's conversion tools (see llama.cpp docs).
5. Start the HTTP server (OpenAI-compatible):

```
cd llama.cpp/build/bin
./llama-server -m /path/to/model.gguf --host 0.0.0.0 --port 8001 --ctx-size
4096
```

6. Optionally enforce an API key:

```
# ./server -m /path/to/model.gguf --host 0.0.0.0 --port 8001 --api-key MYKEY
```

7. In the chat app .env set:

```
LLM_API_BASE=http://localhost:8001/v1
LLM_MODEL=<your model name or leave as placeholder>
LLM_API_KEY=MYKEY   (only if you set --api-key)
```

8. Restart the FastAPI app and test by sending a question in chat.

VI. Self-Hosted Llama 3 Endpoint (Option B)

1. Install vLLM in a Python environment with CUDA (if using GPU).
2. Launch OpenAI-compatible server, e.g.:

```
python -m vllm.entrypoints.openai.api_server --model <your-llama3-model>
--host 0.0.0.0 --port 8001
```

3. Configure the chat app .env similarly to Option A.

VII. Frontend

The frontend is plain HTML/CSS/JS served by FastAPI as static files. It connects via REST for login and message posting, and via WebSocket for real-time message broadcast.

Part 2: Mobile Application Wrapper Interface

I. Android Studio

1. Go to https://developer.android.com/studio to download and install Android Studio
2. While this source has been tested using Android Studio Flamingo, it is expected that the latest version of Android Studio would be able to compile the code.

II. Android Trusted Web Activity (TWA) Application

1. Open the Android project in twa_android_src on Android Studio.
2. Change applicationId/package as needed in app/build.gradle and manifest.
3. Set your deployed web URL in res/values/strings.xml as <string name="defaultUrl">https://YOUR_DOMAIN/</string>
4. Build & run on a device.

III. Asset Links (Required For Full TWA Experience)

1. Generate or use your signing key. Get the SHA-256 fingerprint:

```
keytool -list -v -keystore your_keystore.jks -alias your_alias -storepass <storepass>
-keypass <keypass>
```

2. Copy the SHA-256 fingerprint.
3. On your web server, host `/.well-known/assetlinks.json`. A template is included in the web app at `frontend/.well-known/assetlinks.json`. Replace REPLACE_WITH_YOUR_SHA256_FINGERPRINT.
4. The package name in `assetlinks.json` must match your Android app ApplicationID.
5. After deploying `assetlinks.json`, re-install your signed app and launch. Chrome should open your site as a TWA without URL bar.

IV. Android App

Compile, install, and test your Android App (APK) for your Website on either an Android phone or an Android Emulator

a) LDPlayer: https://www.ldplayer.net
b) BlueStacks: https://www.bluestacks.com/bluestacks-5.html
c) KoPlayer: https://koplayerpc.com/

Deliverables

1. Source code
2. Snapshots of Working Website and Android App
3. Demonstration Video